

Tworzenie dokumentacji projektowej w IT

Dokumentacja techniczna często traktowana jest jako “zło konieczne”. W praktyce jednak, w świecie IT, dobrze napisana dokumentacja to podstawa efektywnej pracy zespołowej, łatwiejszego wdrażania nowych osób w projekt (tzw. onboarding) oraz bezproblemowej komunikacji z klientem. Ten tutorial pokaże Ci, jak efektywnie tworzyć dokumentację techniczną, wykorzystując do tego nowoczesne narzędzia i sztuczną inteligencję.

1. Narzędzia współpracy zespołowej

W profesjonalnych projektach rzadko kiedy piszemy dokumenty lokalnie i wysyłamy je jako załączniki. Podstawą jest praca w chmurze (np. Google Docs), co pozwala na: - **Współpracę w czasie rzeczywistym**: Kilka osób może edytować plik naraz. - **Komentowanie**: Zaznaczanie fragmentów tekstu, aby zgłosić uwagę lub doprecyzować wymagania. W komentarzach można oznaczać innych (np. @jan.kowalski), co od razu przypisuje im zadanie. - **Sugestie**: Tryb sugerowania (odpowiednik śledzenia zmian) jest bardzo podobny do mechanizmu Code Review. Zmiana wejdzie w życie dopiero, gdy ktoś inny ją zatwierdzi.

2. Google NotebookLM – AI do analizy źródeł

Wyobraź sobie, że dostajesz od klienta 200-stronicową specyfikację w PDF, kilkanaście plików tekstowych i wyciąg z bazy danych. Zamiast czytać to wszystko od deski do deski, możesz użyć **Google NotebookLM**.

Jest to narzędzie eksperymentalne od Google, które pozwala na stworzenie własnego “lokalnego” asystenta AI oparty na Twoich źródłach. * **Krok 1**: Importujesz do NotebookLM wszystkie dokumenty projektowe. * **Krok 2**: Narzędzie automatycznie generuje podsumowania każdego źródła. * **Krok 3**: Możesz zadawać pytania typu: “Jakie są wymagania bezpieczeństwa dla modułu płatności?”, a AI wygeneruje odpowiedź, bazując WYŁĄCZNIE na załączonych plikach, podając przypisy do konkretnych akapitów.

3. Markdown - Język dokumentacji w IT

Większość dokumentacji technicznej, szczególnie tej trzymanej w repozytoriach kodu (np. GitHub), powstaje w formacie Markdown. Jest to bardzo prosty język znaczników, który pozwala na szybkie formatowanie tekstu bez odrywania rąk od klawiatury.

Oto najważniejsze elementy składni Markdown:

```
# Nagłówek 1 (Tytuł dokumentu)
## Nagłówek 2 (Rozdział)
### Nagłówek 3 (Podrozdział)
```

****Pogrubienie**** oraz ***kursywa***.

Lista punktowana:

- * Punkt pierwszy
- * Punkt drugi
 - * Podpunkt

Lista numerowana:

1. Pierwszy
2. Drugi

Link: [\[Google\]](https://google.com) (<https://google.com>)

Blok kodu (np. Python):

```
\``python
def hello_world():
    print("Hello, World!")
\``
```

4. Szablony dokumentów: README i User Story

README.md

To pierwszy plik, który widzi osoba przeglądająca Twój kod. Powinien zawierać:
- Nazwę projektu i krótki opis. - Instrukcję instalacji (jak uruchomić projekt na lokalnym komputerze). - Sposób użycia. - Technologie użyte w projekcie.

User Story (Historyjki użytkownika)

To sposób zapisywania wymagań systemowych, bardzo popularny w metodykach zwinnych (Agile/Scrum). Szablon jest prosty: > Jako [typ użytkownika], chcę [wykonać akcję/cel], aby [osiągnąć korzyść/wartość biznesową].

Przykład: “Jako klient sklepu, chcę móc filtrować produkty po cenie, aby szybciej znaleźć asortyment w moim budżecie.”

Podsumowanie

Dokumentacja to narzędzie, a nie cel sam w sobie. Zawsze dostosowuj jej formę (Docs, Markdown) i szczegółowość do odbiorcy – czy będzie to programista rozwijający system, tester sprawdzający funkcjonalności, czy klient biznesowy.